

Corso di Laurea in Ingegneria e Scienze Informatiche

Fancy Title

Tesi di laurea in:
NOME DEL CORSO DEL RELATORE

Relatore

Prof. Supervisor Here

Candidato

Candidate Name

Abstract

Opzionale. Poche righe al massimo.

Contents

Abstract	iii
1 Introduzione	1
1.1 Contesto e motivazione	1
1.2 Obiettivi del progetto	1
1.3 Struttura della tesi	1
2 Background	3
2.1 Sviluppo di applicazioni full stack monolinguaggio	3
2.1.1 Panoramica delle soluzioni esistenti e loro limiti	4
2.1.2 La portabilità e integrazione con librerie native	5
2.2 Kotlin e framework multitarget come risposta al gap	6
2.2.1 Kotlin Multiplatform	6
2.2.2 Interoperabilità con C tramite Cinterop	7
2.3 Lo standard FMI e il formato FMU	8
2.3.1 Functional Mock-up Interface (FMI 2.0)	8
2.3.2 Struttura di un file FMU e modalità operative	8
3 Analisi	9
3.1 Requisiti	9
3.2 Modello del dominio	9
3.2.1 Vincoli	9
4 Design	11
4.1 Architettura	11
4.2 Design di dettaglio	11
4.2.1 Interfacciamento con librerie native	11
4.2.2 Backend multiplatforma	11
4.2.3 Frontend multiplatforma	11

5	Implementazione	13
5.1	Binding C/Kotlin e configurazione del linker per piattaforma	13
5.2	Gestione del ciclo di vita dell'FMU	13
5.3	Estrazione dei metadati e gestione delle variabili	13
5.4	Esecuzione della simulazione	13
5.5	Ricompilazione degli FMU su macOS ARM64	13
5.6	Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma .	13
6	Valutazione	15
6.1	Test unitari e di integrazione	15
6.2	Copertura multiplatforma nella CI	15
7	Conclusioni e Sviluppi Futuri	17
7.1	Limitazioni	17
7.2	Sviluppi futuri	17
7.3	Conclusioni	17
		19
	Bibliography	19

List of Figures

LIST OF FIGURES

List of Listings

LIST OF LISTINGS

Chapter 1

Introduzione

1.1 Contesto e motivazione

1.2 Obiettivi del progetto

1.3 Struttura della tesi

Chapter 2

Background

2.1 Sviluppo di applicazioni full stack monolinguaggio

Prima di addentrarci nella descrizione del background del progetto è importante definire in maniera corretta cosa sono le applicazioni **full stack** e le differenze che emergono tra lo sviluppo di esse e quello di applicazioni più tradizionali. Le componenti delle applicazioni vengono divise in due macrocategorie

- **Frontend:** in questa sezione dell'applicativo sono gestiti tutti gli aspetti inerenti all'interfaccia utente: sia quelli legati all'aspetto grafico che quelli legati alla logica di interazione con essa.
- **Backend:** in questa sezione invece sono gestiti tutti gli aspetti legati all'esecuzione delle funzionalità dell'applicazione, inclusa l'elaborazione dei dati, la loro memorizzazione e la comunicazione con altre applicazioni se necessario.

Le applicazioni *full stack* dunque, comprendono entrambe le componenti. Lo sviluppo di questo tipo di applicazioni può includere l'utilizzo di molteplici linguaggi in base alle necessità, impiegando linguaggi progettati per lo sviluppo del backend e altri per il frontend. Tuttavia questa distinzione al giorno d'oggi viene gradualmente smussata, infatti diversi linguaggi inizialmente progettati per

l'utilizzo in una sola area, vengono poi estesi anche all'altra. L'esempio più rilevante e dominante al giorno d'oggi è **JavaScript**, il quale inizialmente nasce come linguaggio per i browser, quindi dedicato al *frontend*, per poi evolversi oltre i confini del browser permettendone l'esecuzione anche sui server. Questo permette dunque la creazione di applicativi *full stack* monolinguaggio, una tendenza sempre più diffusa. Tra i vantaggi che essa comporta vi sono l'utilizzo di un solo linguaggio per tutto lo stack, il quale porta a una minore curva di apprendimento e una maggiore efficienza per quanto riguarda l'impiego di programmatori. Uno dei più grandi limiti dello sviluppo *full stack* è l'interazione con librerie e Application Programming Interface (API) native, infatti linguaggi come JavaScript sono noti per avere meccanismi limitati o comunque ostici per interagire con il codice nativo. In seguito verranno discusse in dettaglio le soluzioni esistenti per la programmazione di tale tipo di applicativi e i loro limiti, per poi analizzare il problema della portabilità e dell'integrazione con le librerie native.

2.1.1 Panoramica delle soluzioni esistenti e loro limiti

Come anticipato in precedenza, esistono diversi linguaggi impiegati per la programmazione di app *full stack* monolinguaggio. Il più popolare attualmente risulta essere JavaScript [Sta25], questo per via dello sviluppo della tecnologia **Node.js**; infatti questo permette di portare l'ambiente di esecuzione di JavaScript fuori dai browser web. Inoltre anche la creazione di TypeScript, una diretta evoluzione di JavaScript, contribuì alla crescita di questo tipo di linguaggi. Tutti questi fattori rendono JavaScript una delle scelte più gettonate per la creazione di software *full stack*, impiegando diversi stack come Next.js, framework sviluppato da Vercel, e lo stack MEAN, il quale consiste in Mongo DB, Express.js e Angular come framework per il backend e frontend rispettivamente, il tutto legato da Node.js. Nonostante il grande supporto in termini di librerie e framework per questo tipo di tecnologia, l'integrazione con le librerie native risulta comunque ardua per via delle peculiarità del linguaggio. Emerge dunque un limite comune: l'integrazione con librerie native risulta problematica, come verrà analizzato nella sezione seguente.

2.1.2 La portabilità e integrazione con librerie native

Per librerie native si intendono tutte le librerie compilate in linguaggio macchina specifico per un architettura. Queste espongono delle **ABI** (application binary interface) ovvero delle interfacce applicative che comunicano direttamente con il sistema operativo a livello di linguaggio macchina. Tra i linguaggi che vengono compilati in codice macchina figurano, C, C++, Rust e Zig; questi spesso vengono utilizzati per la realizzazione di programmi che hanno necessità di accedere direttamente all'hardware sottostante, oppure per sviluppare librerie che implementano alcuni standard. Quest'ultimo caso è il più rilevante per la tesi, infatti l'applicativo realizzato per questa tesi interagisce con dei file aderenti allo standard Functional Mock-up Interface (FMI), il quale verrà approfondito in seguito nella sezione dedicata.

Diverse delle soluzioni menzionate nella sezione precedente, richiedono la scrittura di codice collante tra il proprio applicativo e le librerie native in caso si vogliano utilizzare. Nel caso più rilevante, ovvero JavaScript, se si necessita di interagire con librerie scritte in C++, è necessario utilizzare una libreria C++ denominata **Node-API**. Tale libreria verrà poi utilizzata per creare moduli che fungono da collante tra JavaScript e la libreria nativa, infatti questo modulo sarà responsabile delle chiamate alla libreria nativa [Nod25]. Questa soluzione permette dunque l'integrazione con questo tipo di librerie, ma contraddice direttamente il principio dietro lo sviluppo *full stack monolinguaggio*, rendendo necessaria la scrittura di codice in linguaggi al di fuori di quello usato per la realizzazione dell'applicativo. Da questa analisi emerge un pattern ricorrente: le soluzioni *full stack monolinguaggio* più mature non sono state costruite attorno all'interoperabilità nativa come requisito principale. Le soluzioni legate a questi linguaggi dunque richiedono di violare il principio del monolinguaggio per interagire con le librerie native. Questo limite non dipende da aspetti implementativi risolvibili con nuove librerie, bensì una conseguenza strutturale basata sulla natura del linguaggio scelto. Dunque una reale soluzione richiederebbe un approccio diverso fin dalla scelta del linguaggio, rendendo necessario l'impiego di un linguaggio **multitarget**.

2.2 Kotlin e framework multitarget come risposta al gap

Alla luce delle limitazioni analizzate nella sezione precedente — in particolare la difficoltà di integrare librerie native C in ambienti JavaScript e la necessità di ricorrere a strati di collante eterolinguistici — questa sezione presenta Kotlin Multiplatform come risposta tecnologicamente motivata a tale gap. L'obiettivo non è descrivere la tecnologia in modo esaustivo, ma illustrare perché le sue caratteristiche architetturali si adattano al problema in esame.

2.2.1 Kotlin Multiplatform

Kotlin nasce nel 2011 come linguaggio sviluppato da JetBrains con l'obiettivo di modernizzare lo sviluppo sulla Java Virtual Machine (JVM): sintassi più concisa, sistema di tipi con null safety integrata e piena interoperabilità con Java esistente [AB⁺20]. La maturità dell'ecosistema e il supporto di JetBrains — nonché l'adozione ufficiale da parte di Google come linguaggio primario per lo sviluppo Android — ne hanno consolidato la credibilità come linguaggio di produzione.

Kotlin Multiplatform (KMP) è un'estensione del compilatore Kotlin che permette di compilare lo stesso codice sorgente verso target eterogenei: JVM, JavaScript per il browser, e codice nativo tramite backend LLVM [Jet24c]. Il punto centrale è che KMP non è un framework di astrazione che interpone un layer uniformizzante tra il codice e la piattaforma: genera artefatti nativi distinti per ciascun target, mantenendo le caratteristiche di performance e interoperabilità proprie di ciascuno. Questa distinzione è rilevante rispetto ad approcci come React Native o Flutter, dove il codice condiviso viene eseguito in un runtime separato.

Il meccanismo che rende possibile la condivisione selettiva del codice è la dichiarazione `expect/actual` [Jet24a]. Nel modulo condiviso (`commonMain`) è possibile dichiarare una funzione o una classe come `expect`, specificandone la firma senza fornirne l'implementazione. Ciascun modulo specifico di piattaforma fornisce poi la corrispondente dichiarazione `actual`, con un'implementazione che può sfruttare le API native del target. Questo meccanismo consente di separare nettamente la logica di business — che risiede nel modulo comune ed è condivisa

tra tutti i target — dalle implementazioni dipendenti dalla piattaforma, senza rinunciare al type checking a tempo di compilazione.

Dal punto di vista del gap identificato in precedenza, la caratteristica rilevante di KMP è la presenza del target **Kotlin/Native**: il compilatore produce binari nativi tramite LLVM, senza dipendenza da una JVM a runtime. Questo apre la possibilità di eseguire codice Kotlin su piattaforme embedded, su macOS e iOS, e in generale in qualsiasi contesto dove un runtime JVM non è disponibile o desiderabile. Soprattutto, Kotlin/Native offre un meccanismo diretto per interfacciarsi con librerie C — aspetto che viene trattato nella sottosezione successiva.

2.2.2 Interoperabilità con C tramite Cinterop

Kotlin/Native include uno strumento dedicato all'interoperabilità con librerie C denominato `cinterop` [Jet24b]. Il suo funzionamento si basa sulla lettura degli header `.h` di una libreria C: a partire da questi, `cinterop` genera automaticamente un modulo Kotlin contenente le definizioni corrispondenti — funzioni, struct, enumerazioni e costanti — con tipi Kotlin appropriati. Lo sviluppatore configura il processo tramite un file `.def`, in cui specifica quali header includere, eventuali flag di compilazione, e parametri di mappatura dei tipi. Il tutto avviene a tempo di compilazione, producendo un modulo che può essere importato nel codice Kotlin come qualsiasi altra dipendenza.

Il confronto con Node-API, descritta nella sezione precedente, è diretto. Con Node-API l'integrazione di una libreria C in un progetto JavaScript richiedeva la scrittura manuale di codice C++ come strato di collegamento: il binding tra i tipi JavaScript e quelli C doveva essere gestito esplicitamente dallo sviluppatore, con tutti i costi di manutenzione e le insidie di type safety che ne derivano. Con `cinterop`, questo strato viene generato automaticamente dal compilatore a partire dagli header: lo sviluppatore scrive esclusivamente in Kotlin, senza mai uscire dal proprio linguaggio di riferimento. Il principio del monolinguaggio — compromesso nell'approccio Node-API — viene quindi preservato.

Vale la pena segnalare un limite architetturale rilevante: `cinterop` è disponibile esclusivamente nel target Kotlin/Native. Il codice che utilizza binding generati da `cinterop` non può quindi risiedere nel modulo `commonMain` di un progetto KMP,

ma deve essere collocato nel modulo nativo specifico. In pratica, questo significa che le chiamate dirette alle librerie C costituiscono un'implementazione **actual** nel modulo nativo, mentre l'interfaccia esposta al codice comune viene dichiarata tramite la corrispondente dichiarazione **expect**. Non si tratta di una limitazione bloccante, ma di un vincolo di design che influenza la struttura del progetto e che verrà ripreso nella descrizione architetturale nei capitoli successivi.

2.3 Lo standard FMI e il formato FMU

2.3.1 Functional Mock-up Interface (FMI 2.0)

2.3.2 Struttura di un file FMU e modalità operative

Chapter 3

Analisi

3.1 Requisiti

3.2 Modello del dominio

3.2.1 Vincoli

Chapter 4

Design

4.1 Architettura

4.2 Design di dettaglio

4.2.1 Interfacciamento con librerie native

4.2.2 Backend multiplatforma

4.2.3 Frontend multiplatforma

Chapter 5

Implementazione

- 5.1 Binding C/Kotlin e configurazione del linker per piattaforma
- 5.2 Gestione del ciclo di vita dell'FMU
- 5.3 Estrazione dei metadati e gestione delle variabili
- 5.4 Esecuzione della simulazione
- 5.5 Ricompilazione degli FMU su macOS ARM64
- 5.6 Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma

Chapter 6

Valutazione

6.1 Test unitari e di integrazione

6.2 Copertura multiplatforma nella CI

Chapter 7

Conclusioni e Sviluppi Futuri

7.1 Limitazioni

7.2 Sviluppi futuri

7.3 Conclusioni

Bibliography

- [AB⁺20] Marat Akhin, Mikhail Belyaev, et al. Kotlin language specification: Kotlin/core. Technical report, JetBrains / JetBrains Research, 2020.
- [Jet24a] JetBrains. Expected and actual declarations. <https://kotlinlang.org/docs/multiplatform-expect-actual.html>, 2024.
- [Jet24b] JetBrains. Interoperability with c. <https://kotlinlang.org/docs/native-c-interop.html>, 2024.
- [Jet24c] JetBrains. Kotlin multiplatform. <https://kotlinlang.org/docs/multiplatform.html>, 2024.
- [Nod25] Node.js. Node-API. <https://nodejs.org/api/n-api.html#node-api>, 2025. Accessed: 2026-06-10.
- [Sta25] Stack Overflow. 2025 stack overflow developer survey. <https://survey.stackoverflow.co/2025/>, 2025. Accessed: 10 June 2026.